

CSCE 701 CYBERSECURITY · MODULE 03 · LECTURE NOTES

Authorisation

Deciding what an authenticated principal may do.

MODULE OUTCOMES

- Distinguish authentication from authorisation precisely.
- Compare the access control models and choose one.
- Explain the confused deputy and the classes it generates.
- Explain why authorisation must be centralised and enforced server-side.
- Review an authorisation design for the standard failures.

1 The distinction, and the models

AUTHENTICATION HAPPENS ONCE; AUTHORISATION HAPPENS ON EVERY REQUEST

Authentication establishes *who* is making the request. **Authorisation** decides *whether this principal may perform this action, on this object, now*. Four parameters, and the third is the one that is dropped.

And the asymmetry in frequency is the operative consequence: **authentication is a single, well-reviewed, heavily-tested code path, and authorisation is a decision that must be made correctly at every endpoint, in every background job, and in every internal tool** — so it is where the misses are.

Which is why **missing or broken authorisation sits consistently near the top of every vulnerability-class ranking** (it is number one in the current OWASP Top Ten), and **why it is fundamentally a coverage problem rather than a design problem**: the design is usually fine and one handler forgot.

So the goal is not a clever access control model but a design in which forgetting the check is *impossible* — **deny-by-default enforcement at a chokepoint, rather than an explicit check written into every handler (§3)**. That reframing from 'remember to check' to 'cannot fail to check' is the main content of this module.

Model	Decides by	The trade
Access control lists	A list of permitted principals attached to each object.	Precise and fine-grained; very hard to answer 'what can Alice do?', which is the question an auditor asks.
RBAC (role-based)	Roles held by the principal, with permissions attached to roles.	Auditable and comprehensible; roles proliferate — see the note.

Model	Decides by	The trade
ABAC (attribute-based)	Attributes of the subject, the object, and the context (time, location, device).	Very expressive; correspondingly hard to reason about and to test, and a policy nobody understands is not a control.
Capability-based	Possession of an unforgeable token that designates both the object and the permitted operation.	No ambient authority, so §2's confused deputy is structurally prevented; revocation and auditing are harder.
ReBAC (relationship-based)	Relationships in a graph — 'is a member of a group that owns the parent folder'.	Fits real sharing models well (it is what Google's Zanzibar implements); needs a dedicated service to answer queries at scale.

RBAC is the sensible default and role proliferation is its characteristic failure — a system with four hundred roles, most held by one person each, has an access control list with extra steps and none of ACL's precision. Which is what pushes organisations toward ABAC or ReBAC, and the honest answer is usually RBAC for coarse decisions plus relationship-based checks for object-level ones.

2 The confused deputy

A CONFUSED DEPUTY is a privileged component that acts on a request without checking whether the REQUESTER had the authority -- using its own authority instead.

THE SAME SHAPE, UNDER MANY NAMES:

CSRF	the browser attaches the user's cookie to a request the attacker caused. The server acts on the user's authority. (Module 06 section 3)
SSRF	your server fetches a URL supplied by a user, from inside your network, using its own network position and identity.
IDOR / missing object check	the handler looks up object 42 because the request said 42, without ever asking whether THIS user may see 42.
PATH TRAVERSAL	the file server opens whatever path it was given, with its own file permissions.
SETUID / SUDO BUGS	the privileged binary acts on attacker-controlled input with root's authority.

THE FIX IS ALWAYS THE SAME SHAPE: the deputy must act on the REQUESTER's authority, not its own. Either pass the authority along explicitly (a capability) or check it against the requester's identity before acting.

AMBIENT AUTHORITY is the underlying cause: the deputy has power merely by virtue of being itself, rather than by having been handed it for this request. Which is exactly why capability systems are structurally resistant.

Ambient authority is the root cause, and recognising the shape means recognising five vulnerability classes at once rather than learning five unrelated acronyms. **It also tells you what to look for in a code review:** any place where a privileged component takes an identifier, a path, or a URL from a less-privileged source and acts on it.

3 Enforcement

AUTHORISATION IS ENFORCED SERVER-SIDE, AT A CHOKEPOINT, DENY-BY-DEFAULT

Server-side, always. A hidden button, a disabled form field, or a client-side role check is a user-interface convenience and not a control — the request can be constructed and sent directly, and will be.

At a chokepoint. A check written into every handler will be omitted from some handler, eventually, by someone in a hurry — so put it in middleware, a policy decision layer, or the data access layer, **where it cannot be skipped by accident**.

Deny by default. A newly added endpoint should be inaccessible until someone explicitly permits it, which **converts 'forgot the authorisation check' from a security vulnerability into a bug report from a confused user** — a dramatically better failure mode, and Module 01 §3's fail-closed principle applied to the development process rather than to runtime.

And object-level, not merely endpoint-level. 'May this user call this endpoint' is a different question from 'may this user access *this particular object*' — and **the second is the one that is missed**, because the first is what a route annotation naturally expresses.

- **Scope queries by principal at the data layer. A query that structurally cannot return another tenant's or another user's rows removes the entire class — the strongest available fix**, and the only one on this list that prevents the mistake rather than catching it.
- **Use unguessable identifiers (UUIDs rather than sequential integers) as defence in depth, and not as the control** — obscurity is not authorisation (Module 01 §3's open design), and identifiers leak through logs, referrers, and shared links.
- **Centralise the policy**, so that it can be read, reviewed, diffed, and tested as a single artefact rather than being distributed across a hundred route handlers.
- **Test authorisation explicitly: for every endpoint, a test asserting that the *wrong* user is refused — which is the test suite nobody writes**, because the positive tests feel like the real ones.
- **And log every authorisation denial**, because **a burst of denials from one principal is among the clearest attack signals you will ever get** (Module 09 §2) — and it is free to collect.

4 Reviewing a design

- **Where is the decision made, and can any code path reach the resource without passing through it?** **Background jobs, admin tools, internal APIs, batch exports, and support consoles are where the second path lives** — and **this question finds the most findings**: an application with impeccable authorisation and an unauthenticated internal admin endpoint is a common and severe pattern.
- **Is the check object-level? Find an endpoint that verifies the role and not the object** — there is usually at least one, and it is usually a report, an export, or a detail view added later.
- **What happens if the policy evaluation itself fails** — the policy service is unreachable, the attribute lookup times out? **Fail closed** (Module 01 §3), and **test that it does**, because the default in most frameworks is not what you want.
- **How is a revocation propagated, and how quickly? A cached authorisation decision is a stale authorisation decision, and the departed employee is the test case**: how long after disabling the account can they still act?
- **And who can change the policy? Authorisation over the authorisation system is the part that is always forgotten**, and it is the highest-value target in the system — a principal who can grant themselves a role has every permission.

MULTI-TENANCY IS THE HARDEST CASE

In a system shared between tenants, every single query must be scoped to the requesting tenant, and one unscoped query is a cross-tenant data exposure — which is the most serious routine failure in software-as-a-service, and it recurs because it requires perfect coverage (§1).

So enforce it structurally rather than by discipline: row-level security in the database, a per-tenant connection or schema, or a data access layer that physically cannot emit an unscoped query — because the discipline approach fails exactly once and that is enough.

And test it adversarially: a generated test suite that exercises every endpoint as tenant A using tenant B's object identifiers, and expects a refusal every single time.

Which is the one test suite in this course worth building purely for its own sake — it is mechanical to generate from your route table, it covers the highest-severity vulnerability class in the system, and almost nobody has one.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OWASP — Authorization and Access Control Cheat Sheets, and the Top Ten A01 (free)	The working reference for §3 and §4 , and A01 documents why this is the top-ranked class.
Hardy — The Confused Deputy (free)	§2's pattern, named — two pages, from 1988, and it explains five modern vulnerability classes.
Pang et al. — Zanzibar: Google's Consistent, Global Authorization System (free)	§1's ReBAC model at scale , and a good illustration of the chokepoint principle implemented as a service.
Saltzer & Schroeder, and Miller's capability work (free)	Least privilege and the capability alternative to ambient authority , which is §2's structural fix.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Classify your system's authorisation** as ACL, RBAC, ABAC, capability, or ReBAC, and name its characteristic failure.
- 2 **Count your roles**. If there are more than twenty, report how many have exactly one holder.
- 3 **Find an instance of each confused-deputy variant** in a deliberately vulnerable training application.
- 4 **Identify every path to one resource** in your system, including jobs and admin tools, and check each passes the authorisation check.
- 5 **Find an endpoint that checks the role and not the object** and fix it.
- 6 **Make your policy evaluation fail** and confirm the system denies rather than permits.
- 7 **Measure revocation latency**: disable an account and time how long until it can no longer act.
- 8 **Scope one query at the data layer** instead of in the handler, and report what it prevented.
- 9 **Generate a cross-tenant test suite** from your route table and run it. Report the failures.
- 10 **Identify who can modify the authorisation policy** and whether that is itself controlled.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish authentication from authorisation, and say why the second is a coverage problem.
- 2 What should the goal be, rather than a clever model?
- 3 Compare five access control models and name RBAC's characteristic failure.

- 4 Define the confused deputy and name five vulnerability classes it covers.
- 5 What is the root cause, and which model resists it structurally?
- 6 Give the three non-negotiables of enforcement and the fourth distinction.
- 7 Which design measure prevents rather than catches?
- 8 Give five review questions, and say which finds the most.
- 9 Why is multi-tenancy hardest, and what is the one test suite worth building?